



Control Hardware in the Loop of Voltage Source Converters (VSCs)

SUNRISE Summer School

Belgrade 10-13/9/2024

Manuel Barragán Villarejo

Department of Electrical Engineering
University of Sevilla

(manuelbarragan@us.es)



This project has received funding from the European Union's Horizon Widening participation and spreading excellence programme under grant agreement No. 101079200





Outline

1. General overview of Typhoon HIL
2. Schematic Editor
3. HIL SCADA
4. Interface Typhoon-HIL 402 to DSP TMS320F28335
5. DSP TMS320F28335
6. Code composer studio
7. Description of the main.c file in CCS
8. Selection of control parameters

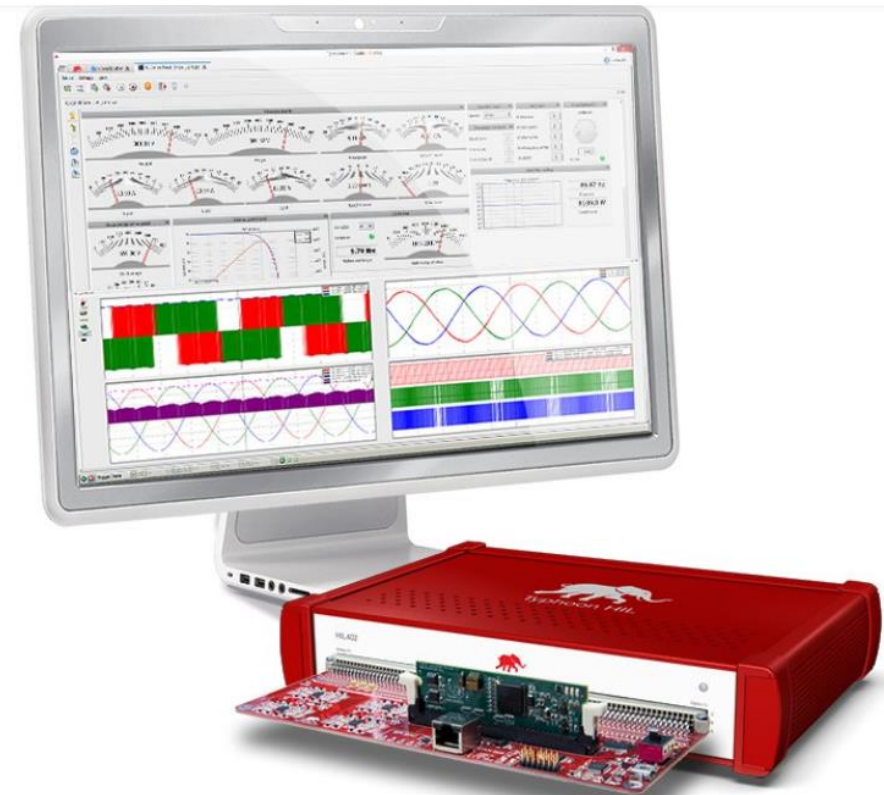


This project has received funding from the European Union's Horizon Widening participation and spreading excellence programme under grant agreement No. 101079200



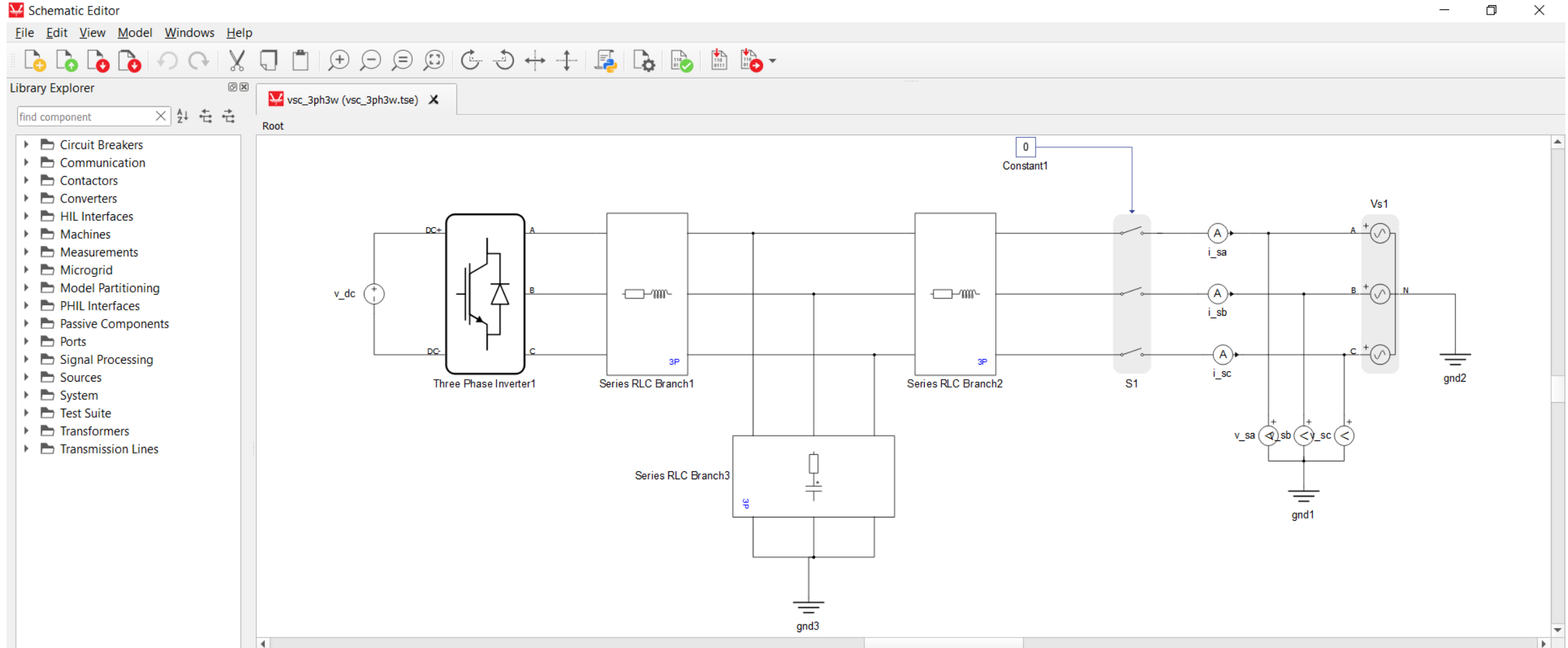
General overview of Typhoon HIL 402/404

- Functionalities:
 - Real time simulation
 - **Control Hardware in the Loop: C-HIL**
 - Power Hardware in the Loop: P-HIL
- Hardware:
 - Real time platform
 - HIL DSP Interface
 - DSP
- Software:
 - Schematic editor
 - HIL Scada
 - Typhoon Test



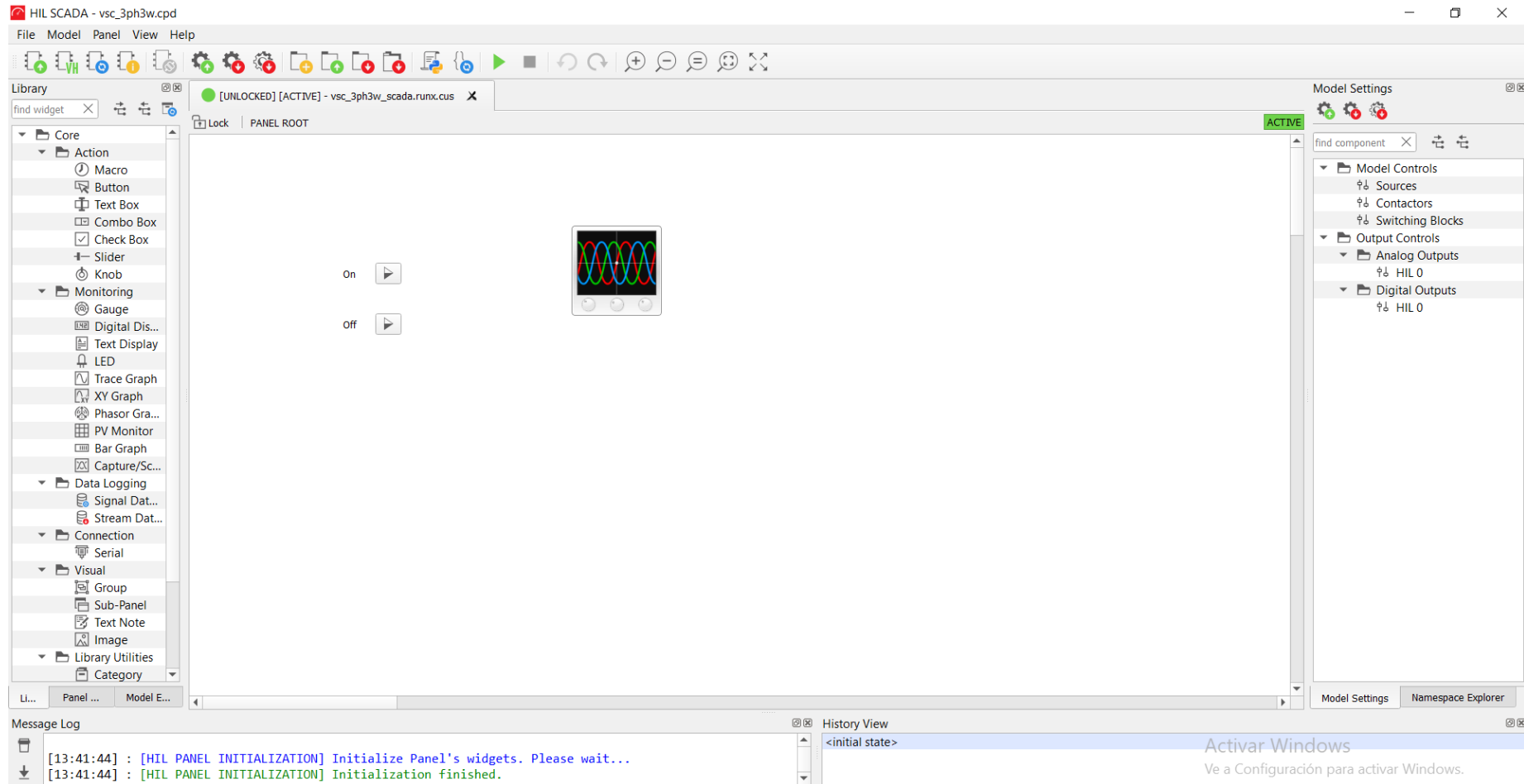
This project has received funding from the European Union's Horizon Widening participation and spreading excellence programme under grant agreement No. 101079200

Schematic Editor



This project has received funding from the European Union's Horizon Widening participation and spreading excellence programme under grant agreement No. 101079200

HIL SCADA



https://www.typhoon-hil.com/documentation/typhoon-hil-api-documentation/typhoon_api.html#

https://www.typhoon-hil.com/documentation/typhoon-hil-software-manual/References/action_widgets.html

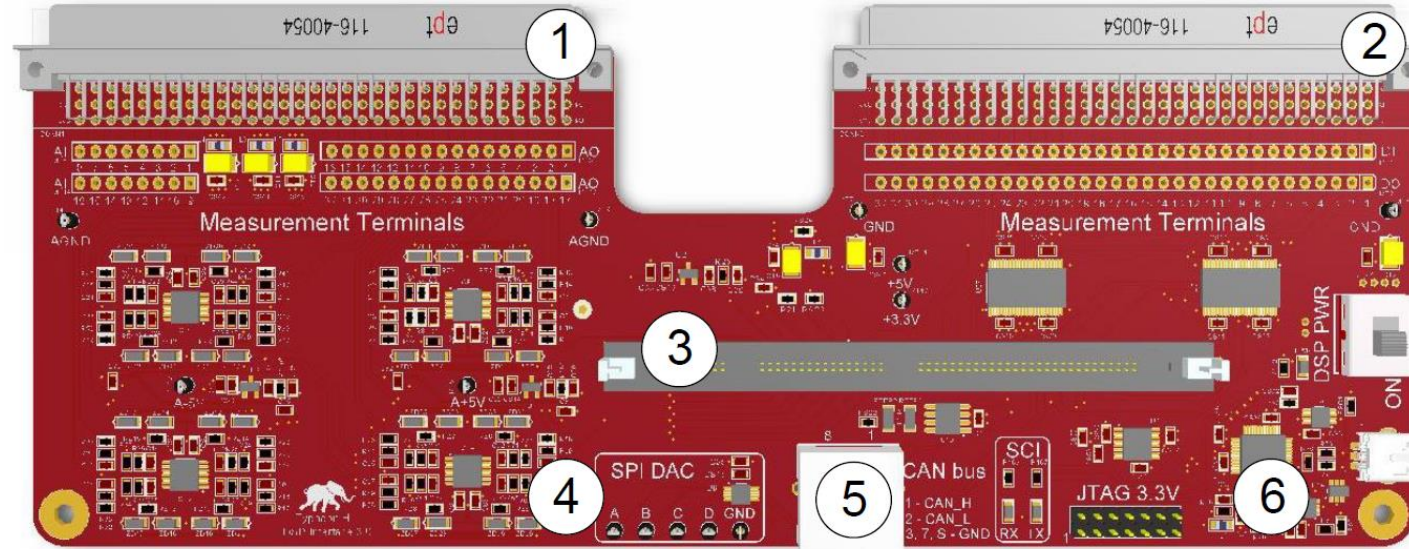


This project has received funding from the European Union's Horizon Widening participation and spreading excellence programme under grant agreement No. 101079200

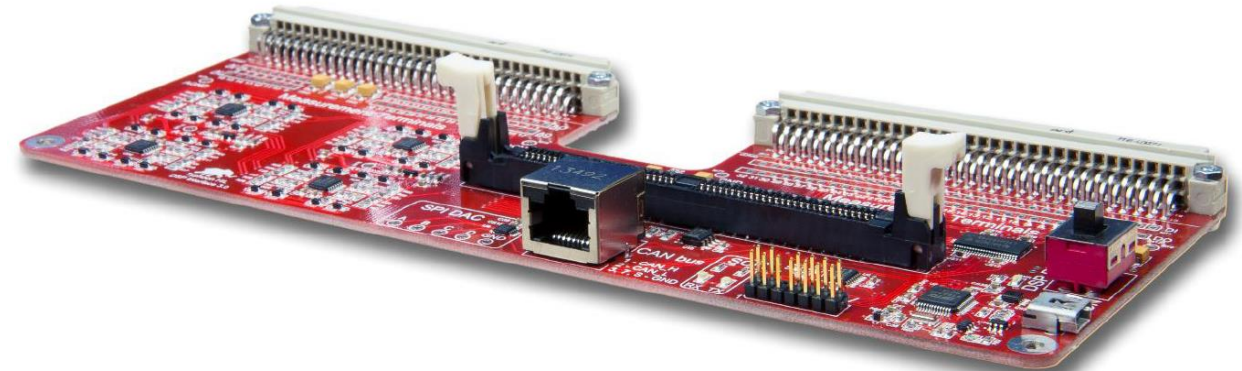


Interface Typhoon-HIL 402/404 to DSP TMS320F28335

- DSP Interface



1. Analog signal connector
2. Digital signal connector
3. DIM100 control card connector
4. SPI (Serial Peripheral Interface) DAC
5. CAN bus
6. XDS100v2 and UART



This project has received funding from the European Union's Horizon Widening participation and spreading excellence programme under grant agreement No. 101079200



Interface Typhoon-HIL 402/404 to DSP TMS320F28335

- DSP Interface
- Analog signal connector

Typhoon HIL Analog Output	DSP Analog Input	
AO1	ADCIN – A0	i_{sc}
AO2	ADCIN – A1	v_{uc}
AO3	ADCIN – A2	i_{uc}
AO4	ADCIN – A3	i_{dc}
AO5	ADCIN – A4	i_{sn}
AO6	ADCIN – A5	
AO7	ADCIN – A6	
AO8	ADCIN – A7	
AO9	ADCIN – B0	v_{dc}
AO10	ADCIN – B1	
AO11	ADCIN – B2	
AO12	ADCIN – B3	v_{sc}
AO13	ADCIN – B4	i_{sb}
AO14	ADCIN – B5	v_{sb}
AO15	ADCIN – B6	i_{sa}
AO16	ADCIN – B7	v_{sa}

Buffer ADC. CCS
03
02
00
09
10
11
01
04
05
06
07
08



Interface Typhoon-HIL 402/404 to DSP TMS320F28335



- DSP Interface
- ## 2. Digital signal connector

Typhoon HIL Digital Output	DSP Digital Input	Typhoon HIL Digital Input	DSP Digital Output
DO1	GPIO-24 / EQEPA-2	DI1	GPIO-00/EPWM-1A
DO2	GPIO-25 / EQEPB-2	DI2	GPIO-02/EPWM-2A
DO3	GPIO-26 / EQEPI-2	DI3	GPIO-04/EPWM-3A
DO4	GPIO-20	DI4	GPIO-06/EPWM-4A
DO5	GPIO-21	DI5	GPIO-08/EPWM-5A
DO6	GPIO-22	DI6	GPIO-10/EPWM-6A
DO7	GPIO-23	DI7	GPIO-01/EPWM-1B
DO8	GPIO-27	DI8	GPIO-03/EPWM-2B
DO9	GPIO-32	DI9	GPIO-05/EPWM-3B
DO10	GPIO-33	DI10	GPIO-07/EPWM-4B
DO11	GPIO-48/40**	DI11	GPIO-09/EPWM-5B
DO12	GPIO-49/41**	DI12	GPIO-11/EPWM-6B
DO13	GPIO-60/44**	DI13	GPIO-14/EPWM-8A*
DO14	GPIO-61/45**	DI14	GPIO-12/EPWM-7A*
DO15	GPIO-62/46**	DI15	GPIO-15/EPWM-8B*
DO16	GPIO-63/47**	DI16	GPIO-13/EPWM-7B*

PWM: Top phase C

PWM: Top DC/DC

PWM: Top phase A

PWM: Top phase B

Interrupt

Enable DC

PWM: Bottom phase C

PWM: Bottom DC/DC

PWM: Bottom phase A

PWM: Bottom phase B

Enable AC





DSP TMS320F28335

- Up to 150 MHz (6.67-ns cycle time)
- On-chip memory: 256K × 16 flash, 34K × 16 SARAM
- Peripherals: 3.3-V I/O design
 - Up to 18 PWM outputs
 - 12-bit ADC, 16 channels
 - Up to 88 individually programmable, multiplexed GPIO pins with input filtering
- Serial port peripherals
 - Up to 2 CAN modules
 - Up to 3 SCI (UART) modules



https://www.ti.com/lit/ds/symlink/tms320f28335.pdf?ts=1725436907130&ref_url=https%253A%252F%252Fdev.ti.com%252F



This project has received funding from the European Union's Horizon Widening participation and spreading excellence programme under grant agreement No. 101079200

Code composer studio



- Code Composer Studio is an integrated development environment (IDE) for TI's microcontrollers and processors. It comprises a suite of tools used to develop and debug embedded applications.
- Includes an optimizing C/C++ compiler, source code editor, project build environment, debugger, profiler and many other features
- Compilers for proprietary architectures such as MSP430™, C2000™ and DSPs are included as well as a compiler for Arm®-based microcontrollers



***Presentation installation_CCS provided contains the details for installation and configuration of CCS**



This project has received funding from the European Union's Horizon Widening participation and spreading excellence programme under grant agreement No. 101079200



Description of the main.c file in CCS

- DSP initialization and configuration is performed
- VSC control functions are executed
- From Line 1 to Line 63.
 - Libraries
 - Functions for configuring peripherals
 - ADC Conversion
 - PLL & Park transformation functions
 - Fault State Machine (FSM)
 - Control functions based on PLL:
 - Control functions based on VSG:
 - Auxiliary computations
 - Global variables

```
main.c ×
1 // Libraries
2 #include "DSP28x_Project.h"
3 #include "type_structs.h"
4 #include <string.h>
5
6 // Constants. Interruptions
7 #define SEQ1_ADC_INT_ENABLE    1
8 #define SEQ2_ADC_INT_ENABLE    1
9
10
11 // Peripherals
12 interrupt void adc_SEQ1_isr(void);           // ADC Interrupt SEQ1
13 interrupt void adc_SEQ2_isr(void);           // ADC Interrupt SEQ2
14 void Setup_ePWM(void);                       // ePWM Configuration
15 void Setup_DMA(Uint16 *ADC_read, Uint16 *duty); // DMA Configuration
16 void Setup_ADC(void);                         // ADC Configuration
```





Description of the main.c file in CCS

- From Line 66 to Line 141. **'main'** function
 - DSP Initialization
 - Initialization of the flash memory
 - Clock frequency definition
 - GPIOs Initialization and Configuration
 - Interrupts Initialization
 - DMA Initialization
 - ADC Initialization
 - ePWM Initialization

```
66 // Main function
67 void main(void)
68 {
69     // DSP Initialization
70     InitSysCtrl();
71
72     EALLOW;
73     SysCtrlRegs.MAPCNF.bit.MAPEPWM = 1; // Enables ePWM remap to Peripheral Frame 3 (accessible by DMA)
74     EDIS;
75
76     // Location of the memory from the flash to the RAM
77     memcpy(&RamfuncsRunStart, &RamfuncsLoadStart, &RamfuncsLoadEnd - &RamfuncsLoadStart);
78
79     // Initialization of the flash memory
80     InitFlash();
```





Description of the main.c file in CCS

- From Line 143 to Line 151. **ADC interruption 1 for ADC**
- From Line 154 to Line 225. **ADC interruption 1 for control loop**
 - GIP012 for enabling switching the VSC
 - Calibration. Input ADC_read. Analog channels reading
 - RMS computation. Input ADC converts to Volts and Amps
 - power computation
 - PLL and Park transformation
 - **State machine**
 - Enable/disable ePWM
 - **Controller selection**
 - Carrier based PWM
 - Reinitialize for next ADC sequence

```
154 // ADC Interrupt 2
155 #pragma CODE_SECTION(adc_SEQ2_isr, "ramfuncs")
156 interrupt void adc_SEQ2_isr(void)
157 {
158
159 // Typhoon activates GIP012 for enabling switching the VSC
160 #if DEVICE == 4
161     if(GpioDataRegs.GPADAT.bit.GPIO12==0){GpioDataRegs.GPASET.bit.GPIO12=1;}
162     else{GpioDataRegs.GPACLEAR.bit.GPIO12=1;}
163 #endif
164
165 // Calibration. Input ADC_read. Analog channels reading
166     ADC_processed = Conversion_ADC(ADC_read);
167
```



Description of the main.c file in CCS



- File 'State_machine.c'. Function state_mac. Table with type of Faults

Value	Type of fault
0	There is no type of <u>fault</u> and the system enables the VSC to operate
1	Overcurrent in the current injected by the VSC to the grid
2	Grid <u>Undervoltages</u>
3	Grid <u>Overvoltages</u>
4	Grid <u>Underfrequency</u>
5	Grid <u>Overfrequency</u>
6	IGBTs driver fault
7	
8	
9	Waiting to enable the state machine
10	Bus DC <u>undervoltage</u>
11	Bus DC overvoltage
12	DC/DC converter <u>undervoltage</u>
13	DC/DC converter overvoltage
14	DC/DC converter overcurrent





Description of the main.c file in CCS

- **Controller selection**

```
// controller selection
// #if (CTRL == 2) // dq current controller based on PLL
//     eta=DQCURRENT_comp(ADC_processed, PLL_Park, setpoints.id_ref, setpoints.iq_ref, state.PWM_enabled && setpoints.start_AC);
//
// #elif (CTRL == 3) // dq power controller based on PLL
//     eta=DQPOWER_comp(ADC_processed, PLL_Park, setpoints.p_ref, setpoints.q_ref, state.PWM_enabled && setpoints.start_AC);
//
// #elif (CTRL == 11) // dq statcom controller based on PLL
//     eta=DQSTATCOM_comp(ADC_processed, PLL_Park, setpoints.vdc_ref, setpoints.q_ref, state.PWM_enabled && setpoints.start_AC);
//
// #endif
//
// #if (CTRL == 5) // dq VSG based on PI controller
//     uvsg_out = VSG_comp(setpoints.p_ref, setpoints.q_ref, ADC_processed, PLL_Park, state.PWM_enabled && setpoints.start_AC);
//     eta=DQCURRENT_comp(ADC_processed, PLL_Park, uvsg_out.i_d_ref, uvsg_out.i_q_ref, state.PWM_enabled && setpoints.start_AC);
//
// #endif
```

Function DQCURRENT_comp → dq current controller

Function DQPOWER_comp → dq power controller

Function DQSTATCOM_comp → dq statcom controller

Function VSG_comp → dq vsg controller





Selection of control parameters

- Header file: type_structs.h
 - PWM timer
 - Dead-band PWM
 - Execution rate of the controller
 - Device to which the dsp connects
 - Number of wires of the VSC
 - **Controller selection**
 - Synchronization of the DSP
 - Enabling the state machine
 - ADC calibration
 - Limits of operation of the VSC
 - Controller gains
 - Structures: They are a way to group several related variables into one place

```
1 #define PWM_TIMER_TBPRD      3750    // 20 kHz -> 50 us
2 #define EPWM_MIN_DB          8        // Dead-band of 0.1us for ePWM (Table 3.15)
3 #define Ts                    50e-6    // Execution rate of the microcontroller
4 #define DEVICE                4        // Device number for ADC calibration parameters
5     // DEVICE    4            TYPHOON
6
7 #define wires                 3 //number of operative wires into the VSC
8 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
9 #define CTRL                  2
10 /* CTRL code
11     2 -> dq current controller
12     3 -> active an reactive power controller
13     11 -> STATCOM. DC bus voltage controller and reactive power controller
14     5 -> Grid-Forming PI-VSG (PQ in W/var) */
15
16 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
17
18 #define EXT_SYNC              0        // EXT_SYNC = 0 -> EXTERNAL SYNC OFF (MA
19 #define FSM                   1        // Enable FSM
```





Outline

1. General overview of Typhoon HIL
2. Schematic Editor
3. HIL SCADA
4. Interface Typhoon-HIL 402 to DSP TMS320F28335
5. DSP TMS320F28335
6. Code composer studio
7. Description of the main.c file in CCS
8. Selection of control parameters





Control Hardware in the Loop of Voltage Source Converters (VSCs)

SUNRISE Summer School

Belgrade 10-13/9/2024

Manuel Barragán Villarejo

Department of Electrical Engineering
University of Sevilla

(manuelbarragan@us.es)



This project has received funding from the European Union's Horizon Widening participation and spreading excellence programme under grant agreement No. 101079200

